

Local GPU-Accelerated LLM Inference PoC

Containerized serving, GPU validation, and load benchmarking on an NVIDIA RTX 4060 8 GB

Prepared by	Kazi Md Samir
Email	kzsamir849@gmail.com
Phone	01884382430
Repository	github.com/kzmdsamir/llm-gpu-docker-mini

Built a reproducible local inference workflow: deploy an API container, validate Docker-to-GPU access, exercise the serving path under load, preserve benchmark evidence, and turn observed limits into a concrete improvement plan.

Executive Summary

This project is a local, GPU-backed inference proof of concept aligned to the Solar GPU PoC AI Infrastructure Internship. It packages a FastAPI gateway in Docker, uses host Ollama as the model-serving runtime, adds health and readiness checks, emits structured JSON logs, and includes smoke tests plus concurrent benchmark automation.

The intent is not to claim a production deployment or a solar-powered cluster. The intent is to demonstrate practical systems work: repeatable deployment, GPU verification, controlled measurement, honest interpretation, and well-documented next steps.

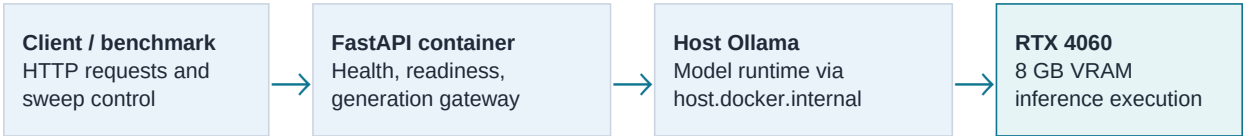
What Was Delivered

FastAPI endpoints for liveness, readiness, and generation; environment-driven configuration; structured request logs; Dockerfile and Compose deployment; automated smoke tests; concurrent Python load testing; CSV and JSON result artifacts; and an experiment template for repeatable engineering notes.

Project Objective

Measure how an RTX 4060 8 GB behaves while serving the Ollama model tag qwen3.5:9b at increasing request concurrency, then identify the operational bottlenecks relevant to a larger shared GPU environment.

System Design and Validation



Service Design

The FastAPI container exposes GET /health for application liveness, GET /ready to test upstream Ollama reachability, and POST /generate to forward prompts and return latency, generated token count, generation rate, and a request ID. JSON logs preserve generation start, completion, and upstream failure events.

Deployment Decision

The default Compose path runs Ollama on the host and connects from the container through host.docker.internal. GPU execution therefore occurs in the host Ollama process. The API container has an NVIDIA GPU device request configured, but it is not required while the container only relays requests. GPU allocation becomes required when the model runtime is moved into Docker.

Verified Environment

Host	Windows 11 with WSL2 Ubuntu and Docker Desktop
CPU / RAM	AMD Ryzen 7 7700, 8 cores / 32 GB RAM
GPU	NVIDIA GeForce RTX 4060, 8,188 MiB VRAM
Driver / Docker	NVIDIA 596.49 / Docker Engine 28.5.1
Serving runtime	Host Ollama via host.docker.internal:11434
Model	qwen3.5:9b (quantization metadata not recorded)

Independent GPU proof: docker run --rm --gpus all nvidia/cuda:12.2.0-base-ubuntu22.04 reported NVIDIA GeForce RTX 4060, driver 596.49, and 8188 MiB VRAM from inside the container.

Benchmark Evidence and Interpretation

Method

A Python ThreadPoolExecutor issued 20 HTTP requests to POST /generate with the prompt Explain Docker in 2 sentences. The benchmark records request latency, returned token count, and the per-request generation rate reported by the API, then writes CSV and JSON artifacts. The initial workload intentionally had no MAX_TOKENS cap.

Saved Raw Results

Workers	Success	Mean	P50	P90	P99	Gen tok/s	Outcome
1	19/20	45.9 s	41.0 s	84.3 s	93.2 s	24.7	Completed; one failure
2	19/20	95.5 s	80.2 s	158.1 s	175.0 s	13.3	Completed; one failure
4	N/A	-	-	-	-	-	Saturated; interrupted

The worker-specific JSON summaries are treated as the source of truth. An earlier experiment note reports 20/20 success at two workers, while the saved worker-1 and worker-2 summaries each record 19/20 success. The mismatch is documented as a report-quality issue to correct rather than concealed.

Key Findings

1. GPU-container support is verified independently from application behavior. 2. The 9B model is usable at limited concurrency, but mean latency more than doubles from one to two workers. 3. Tail latency and failure risk increase under concurrent load. 4. The four-worker sweep saturated the system and was interrupted. 5. Unbounded generation was the dominant source of response-time variation.

Why the Latency Is High

Although the prompt asked for two sentences, successful worker-2 requests generated approximately 598 to 3,216 tokens. This test is useful as a stress observation, but it is not a controlled interactive-latency baseline. The next run should cap output at 128 or 256 tokens and include output-token counts in every summary.

Metric Discipline

The reported tokens/s value is a per-request generation-rate observation, not total system throughput. Future reports should add requests/s and total generated tokens divided by full test wall time. With only 19 successful requests, P95 is more useful than P99; percentile calculation should use a nearest-rank method before formal comparison.

Operational Lessons and Internship Relevance

Observed Constraints

The application logs recorded upstream Ollama read timeouts under heavier load. The 180-second timeout protects callers from waiting indefinitely, but it does not provide backpressure. The benchmark CSV also omits exception text for failed rows, and GPU utilization telemetry was observed with nvidia-smi rather than stored as a time series. These are practical improvements, not hidden weaknesses.

Immediate Improvement Plan

Repeat the worker-1 and worker-2 runs with MAX_TOKENS=128 and 256; record model metadata with ollama show; persist error classes in CSV; add queue or concurrency limits; capture GPU memory, utilization, temperature, CPU, and RAM during each run; and compare a smaller model using the same fixed workload.

Direct Relevance to the Internship

Internship focus	Evidence from this project
Containerized AI	Dockerfile, Compose, health checks, host-runtime networking
Inference serving	Ollama-backed FastAPI generation API and readiness checks
GPU operations	Host and CUDA-container GPU validation on RTX 4060
Benchmarking	Concurrent load test, raw CSV/JSON, worker sweep, overload evidence
Observability	Structured request logs, request IDs, nvidia-smi monitoring workflow
Automation / docs	Smoke test, benchmark wrappers, experiment template, repeatable report

Portfolio statement: I can take a local AI inference service from deployment through verification and measurement, identify reliability limits honestly, and translate results into the next operational experiment for a larger GPU PoC.

Reproduce the Baseline

ollama list | docker compose up --build -d | ./scripts/smoke_test.sh | nvidia-smi -l 1 | MAX_WORKERS=1 MAX_TOKENS=256 python scripts/benchmark.py

Source evidence: app/main.py, docker-compose.yml, scripts/benchmark.py, scripts/smoke_test.sh, results/benchmark_results.csv, results/benchmark_summary_workers_1.json, results/benchmark_summary_workers_2.json, and docs/experiment_01_qwen3_5_9b_gpu.md.